Aide-mémoire JavaScript

> Tip
>
> **Fait par :** Paul Friedli
> **Version :** 26.05.2025
> **Référence :** https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference
> **Tester du code JS** : https://playcode.io/empty_javascript
> **Convertir en PDF** : https://marketplace.visualstudio.com/items?itemName=manuth.markdown-converter

Table des matières

- `console.table()`
- Parcourir un tableau
  - `forEach` - parcourir les éléments d'un tableau
  - `entries()` - parcourir les couples index/valeurs d'un tableau
  - `in` - parcourir les clés d'un tableau
  - `of` - parcourir les valeurs d'un tableau
- `find()` - premier élément qui satisfait une condition
- `findIndex()` - premier index qui satisfait une condition
- `indexOf()` et `lastIndexOf()` - premier/dernier élément qui correspond
- `push()`, `pop()`, `shift()` et `unshift()` - ajouter/supprime au début/fin dans un tableau
- `slice()` - ne conserver que certaines lignes d'un tableau
- `splice()` - supprimer/insérer/remplacer des valeurs dans un tableau
- `concat()` - joindre deux tableaux
- `join()` - joindre des chaînes de caractères
- `keys()` et `values()` - les clés/valeurs d'un objet
- `includes()` - vérifier si une valeur est présente dans un tableau
- `every()` et `some()` - vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau
- `fill()` - remplir un tableau avec des valeurs
- `flat()` - aplatir un tableau
- `sort()` - pour trier un tableau
- `map()` - tableau avec les résultats d'une fonction
- `filter()` - tableau avec les éléments passant un test
- `groupBy()` - regroupe les éléments d'un tableau selon un règle
- `flatMap()` - chaînage de map() et flat()
- `reduce()` et `reduceRight()` - réduire un tableau à une seule valeur
- `reverse()` - inverser l'ordre du tableau
- `...` - l'opérateur 'spread' pour décomposer un tableau/objet en ses éléments
- `split()` - un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau

- `trim()`, `trimStart()` et `trimEnd()` - épuration des espaces en trop dans une chaîne (trimming)
- `JSON.stringify()` - transformer un objet Javascript en JSON
- `JSON.parse()` - transformer du JSON en objet Javascript
- `padStart()` et `padEnd()` - aligner le contenu dans une chaîne de caractères
- `` `` (backticks) - pour des expressions intelligentes
- `new Set()` - pour supprimer les doublons
- Déclaration de fonction
- Fonctions immédiatement invoquées (IIFE) et lambda expressions
- Utilisation combinée de `map()` + `filter()` + `reduce()` + autres
- Asynchronisme avec `async`, `await` et les `promises`

`console.table()`

Affiche **un tableau** ou **un objet** de la manière la plus lisible possible sur la console (voir documentation officielle pour des exemples visuels).

```
console.table(["apples", "oranges", "bananas"]);
```

(index)	Values
0	'apples'
1	'oranges'
2	'bananas'

```
const him = new Person("Tyrone", "Jones");
console.table(him);
```

(index)	Values
firstName	'Tyrone'
lastName	'Jones'

Parcourir un tableau

`forEach` - parcourir les éléments d'un tableau

Parcourir tous les éléments d'un tableau.

```
const arr = ['a', 'b', 'c'];

// Manière fonctionnelle - impossible d'utiliser return ou break !
arr.forEach((val, index, arr) => { console.log(val, index); });
// a 0, b 1, c 2
```

`entries()` - parcourir les couples index/valeurs d'un tableau

Parcourir tous les éléments d'un tableau.

```
const arr = ['a', 'b', 'c'];

// style for loop
for (const [index, val] of arr.entries()) { console.log(val, index);
}  // a 0, b 1, c 2
```

`in` - parcourir les clés d'un tableau

Parcourir toutes les clés d'un tableau.

```
const arr = ['a', 'b', 'c'];

// boucle for in
for (const key in arr) { console.log(key); }  // 0, 1, 2
```

`of` - parcourir les valeurs d'un tableau

Parcourir toutes les valeurs d'un tableau.

```
const arr = ['a', 'b', 'c'];

// boucle for of
for (const value of arr) { console.log(value); }  // a, b, c
```

`find()` - premier élément qui satisfait une condition

Retourne le premier élément d'un tableau satisfaisant une condition ou `undefined` sinon.

```
const foundElement = elements.find((element) => element.name ===
'b');
```

`findIndex()` - premier index qui satisfait une condition

Retourne l'index du premier élément satisfaisant une condition ou `-1` sinon.

```
const foundElementIndex = elements.findIndex((element) =>
element.name === 'b');
```

`indexOf()` et `lastIndexOf()` - premier/dernier élément qui correspond

Retourne l'indice du premier/dernier élément correspondant à une valeur ou `-1` sinon.

```
const foundElementIndex = elements.indexOf('James');
const foundElementIndex = elements.indexOf('James', 3); // Commence
la recherche au 4e élément
```

`push()`, `pop()`, `shift()` et `unshift()` - ajouter/supprime au début/fin dans un tableau

- `push()` ajoute un élément à la fin du tableau.
- `pop()` supprime le dernier élément du tableau.
- `unshift()` ajoute un élément au début du tableau.
- `shift()` supprime le premier élément du tableau.

```
const array1 = [1, 2, 3];
array1.unshift(4, 5);
console.log(array1); // sortie attendue : Array [4, 5, 1, 2, 3]
```

`slice()` - ne conserver que certaines lignes d'un tableau

Renvoie une copie superficielle (shallow copy) d'une portion du tableau d'origine, définie par un indice de début (compris) et un indice de fin (exclus).

N.B. Si fin est un indice négatif, il indique un décalage depuis la fin de la séquence.

```
const animals = ["ant", "bison", "camel", "duck", "elephant"];

console.log(animals.slice(2));
// Expected output: Array ["camel", "duck", "elephant"]

console.log(animals.slice(2, 4));
// Expected output: Array ["camel", "duck"]

console.log(animals.slice(1, -1));
// Expected output: Array ["bison", "camel", "duck"]
```

`splice()` - supprimer/insérer/remplacer des valeurs dans un tableau

Supprime ou ajoute des éléments dans un tableau.

```
var months = ['Jan', 'March', 'April', 'June'];

months.splice(1, 0, 'Feb'); // insertion à l'indice 1
console.log(months);  // ["Jan", "Feb", "March", "April", "June"]

months.splice(4, 1, 'May1', 'May2'); // remplace 1 élément à l'indice
4
console.log(months);  // ["Jan", "Feb", "March", "April", "May1",
"May2"]
```

`concat()` - joindre deux tableaux

Joint deux tableaux sans modifier les originaux.

```
const data1 = [1, 2, 3];
const data2 = [10, 20, 30, 40];
const data3 = data1.concat(data2);  // data3 = [1, 2, 3, 10, 20, 30,
40]
```

`join()` - joindre des chaînes de caractères

Retourne une chaîne en concaténant tous les éléments du tableau avec un séparateur.

```
const data = [1, 2, 3];
const values = data.join(', '); // '1, 2, 3'
```

`keys()` et `values()` - les clés/valeurs d'un objet

Retourne toutes les clés ou toutes les valeurs d'un objet.

```
const object1 = {
  a: "somestring",
  b: 42,
  c: false,
};

console.log(Object.keys(object1));  // ["a", "b", "c"]
console.log(Object.values(object1)); // ["somestring", 42, false]
```

`includes()` - vérifier si une valeur est présente dans un tableau

Retourne `true` si la valeur existe dans le tableau.

```
const data = [1, 2, 3];
const isValueInData = data.includes(2);
```

`every()` et `some()` - vérifier si plusieurs valeurs sont toutes/quelques présentes dans un tableau

- `every()` retourne `true` si chaque élément passe le test.
- `some()` retourne `true` si au moins un élément passe le test.

```
const data = [1, 2, 3];
const isAllValuesDigits = data.every(elem => (elem < 10) &&
(elem>=0)); // true
```

`fill()` - remplir un tableau avec des valeurs

Remplit un tableau avec des valeurs.

```
const data = [1, 2, 3, 4];

// fill with 0 from position 2 until position 4
const newData = data.fill(0, 2, 4);  // [1, 2, 0, 0]

// fill with 0 from position 2 until position 4
const newData = data.fill(6);  // [6, 6, 6, 6]
```

`flat()` - aplatir un tableau

Aplatit un tableau.

```
const twoLevelsDeep = [[1, [2, 2], 1]];

twoLevelsDeep.flat();       // [1, [2, 2], 1]
twoLevelsDeep.flat(2);       // [1, 2, 2, 1]

const veryDeep = [[1, [2, 2, [3,[4,[5,[6]]]]], 1]];
veryDeep.flat(Infinity);      // [1, 2, 2, 3, 4, 5, 6, 1]
```

`sort()` - pour trier un tableau

Trie un tableau.

```
let data = [1, 2, 3, 4];
data.sort();

let items = [
  { name: "Edward", value: 21 },
  { name: "Sharpe", value: 37 },
  { name: "And", value: 45 },
  { name: "The", value: -12 },
  { name: "Magnetic", value: 13 },
  { name: "Zeros", value: 37 }
];
items.sort(function (a, b) {
  return a.value - b.value;
});
```

`map()` - tableau avec les résultats d'une fonction

Crée un tableau avec les résultats d'une fonction.

```
const newDataArray = dataArray.map((item) => performSomething(item))

const numbers = [2, 4, 8, 10];
const halves = numbers.map(x => x / 2); // [1, 2, 4, 5]
```

`filter()` - tableau avec les éléments passant un test

Crée un tableau avec les éléments passant un test.

```
const newDataArray = dataArray.filter((item) => performTest(item))

const words = ["exuberant", "spray", "elite", "destruction", "limit",
"present"];
const longWords = words.filter(word => word.length > 6);    //
longWords is ["exuberant", "destruction", "present"]
```

`groupBy()` - regroupe les éléments d'un tableau selon un règle

> ⚠ Warning
>
> **ATTENTION ⚠ :** Opérateur **génial** mais disponible depuis peu (>= mars 2024 !!)

Regroupe et ventile les éléments d'un tableau selon une méthode fournie.

```
const inventaire = [
        { produit:"asperges",    categorie:"legume",   nbre:9 },
        { produit:"bananes",     categorie:"fruit",    nbre:5 },
        { produit:"poulpe",      categorie:"poisson",  nbre:3 },
        { produit:"jambon cru",  categorie:"viande",   nbre:9 },
        { produit:"fraises",     categorie:"fruit",    nbre:15 },
        { produit:"cotelette",   categorie:"viande",   nbre:14 },
        { produit:"cerises",     categorie:"fruit",    nbre:22 },
        { produit:"truite",      categorie:"poisson",  nbre:12 },
        { produit:"filet angus", categorie:"viande",   nbre:11 },
        { produit:"carotte",     categorie:"legume",   nbre:62 }
];

const inventaireParGroupes = Object.groupBy(inventaire, ({ nbre }) =>
  nbre < 0 ? "racheter" : "suffisant",
);

console.log(JSON.stringify(inventaireParGroupes));
```

La sortie console sera :

```
{
    "racheter" : [
        { "produit":"asperges",    "categorie":"legume",   "nbre":9
},
        { "produit":"bananes",     "categorie":"fruit",    "nbre":5
},
        { "produit":"poulpe",      "categorie":"poisson",  "nbre":3
},
        { "produit":"jambon cru",  "categorie":"viande",   "nbre":9 }
    ],
    "suffisant" : [
        { "produit":"fraises",     "categorie":"fruit",    "nbre":15
},
        { "produit":"cotelette",   "categorie":"viande",   "nbre":14
},
        { "produit":"cerises",     "categorie":"fruit",    "nbre":22
},
        { "produit":"truite",      "categorie":"poisson",  "nbre":12
},
        { "produit":"filet angus", "categorie":"viande",   "nbre":11
},
        { "produit":"carotte",     "categorie":"legume",   "nbre":62
}
    ]
}
```

`flatMap()` - chaînage de map() et flat()

Correspond à l'enchaînement de map() suivi de flat() avec une profondeur 1.
Cela permet donc d'appliquer une fonction à chaque élément du tableau puis d'aplatir le résultat en un tableau.

```
let tableau1 = ["Coucou comment", "", "ça va ?"];

tableau1.map((x) => x.split(" "));
// [["Coucou", "comment"], [""], ["ça", "va", "?"]]

tableau1.flatMap((x) => x.split(" "));
// ["Coucou", "comment", "", "ça", "va", "?"]
```

`reduce()` et `reduceRight()` - réduire un tableau à une seule valeur

Réduit un tableau à une seule valeur en appliquant une fonction (qui est comme un « accumulateur ») qui traite chaque valeur d'une liste (de la gauche vers la droite) afin de la réduire à une seule valeur. Une valeur initiale peut être fournie.

```
const total = [0, 1, 2, 3];
const result = total.reduce((sum, value) => sum + value, 1); //
result is 7
```

`reverse()` - inverser l'ordre du tableau

Inverse l'ordre du tableau.

```
const data1 = [0, 1, 2, 3];
const data2 = data1.reverse(); // [3, 2, 1, 0]
```

`...` - l'opérateur 'spread' pour décomposer un tableau/objet en ses éléments

L'opérateur de décomposition spread `...` permet de décomposer un itérable (comme un tableau) en plusieurs éléments distincts. Cela nous permet de copier rapidement tout ou une partie d'un tableau existant dans un autre tableau ou d'en extraire facilement des parties.

```
// Combiner des valeurs existantes dans un nouveau tableau
const numbersOne = [1, 2, 3];
const numbersTwo = [4, 5, 6];
const numbersCombined = [...numbersOne, ...numbersTwo];

// Extraire uniquement ce qui est utile d'un tableau
const numbers = [1, 2, 3, 4, 5, 6];
const [one, two, ...rest] = numbers;

// Mariage d'objets avec mise à jour :-)
const myVehicle = {
  brand: 'Ford',
  model: 'Mustang',
  color: 'red'
}
const updateMyVehicle = {
  type: 'car',
  year: 2021,
  color: 'yellow'
}
const myUpdatedVehicle = {...myVehicle, ...updateMyVehicle}
```

`split()` - un ciseau qui coupe une chaîne là où un caractère apparaît et produit un tableau

`split()` divise une chaîne de caractères en un tableau de sous-chaînes. Une sous-chaîne est créée là où le motif fourni est présent.

```
const str = "The quick brown fox jumps over the lazy dog.";
const words = str.split(" ");   // words = ['The', 'quick', 'brown',
'fox', 'jumps', 'over', 'the', 'lazy', 'dog.'];

const str2 = "Un##simple#exemple";
const words2 = str2.split("#");   // words2 = ["Un", "", "simple",
"exemple"];
```

`trim()`, `trimStart()` et `trimEnd()` - épuration des espaces en trop dans une chaîne (trimming)

`trim()`, `trimStart()` et `trimEnd()` pour supprimer les éventuels espaces, tabulations, ... au début et/ou à la fin d'une chaîne de caractères.

```
const greeting = "   Hello world!   ";
console.log(greeting.trim());           // Expected output: "Hello
world!";
console.log(greeting.trimStart());      // Expected output: "Hello
world!   ";
```

`JSON.stringify()` - transformer un objet Javascript en JSON

`JSON.stringify(obj)` ou `JSON.stringify(obj, null, ' ')` permet de transformer tout objet Javascript en sa représentation JSON.

```
console.log( JSON.stringify( { x: 5, y: 6 } ) );
// Expected output: '{"x":5,"y":6}'

const obj = [{ x: 5}, true, "coucou", { y: 6 }];
console.log( JSON.stringify( obj ) );
// Expected output: '[{"x":5},true,"coucou",{"y":6}]'

console.log( JSON.stringify( obj , null, '    ') );
// Expected output:
// [
//     {
//         "x": 5
//     },
//     true,
//     "coucou",
//     {
//         "y": 6
//     }
// ]
```

`JSON.parse()` - transformer du JSON en objet Javascript

`JSON.parse()` permet de transformer une chaîne de caractère JSON en son objet Javascript correspondant.

```
const strJSON = '{"Name":"GFG","Age":22,"Department":"Computer
Science and Engineering","Year":"3rd"}';
const obj = JSON.parse(strJSON);

// obj will now contain this :
const sameObj = {
   Name: "GFG",
   Age : 22,
   Department : "Computer Science and Engineering",
   Year: "3rd"
};
```

`padStart()` et `padEnd()` - aligner le contenu dans une chaîne de caractères

`padStart()` et `padEnd()` pour aligner le contenu de chaînes de caractères.

```
const str1 = "5";
console.log(str1.padStart(4, " "));         // Expected output: "
5"

const fullNumber = "2034399002125581";
const last4Digits = fullNumber.slice(-4);
const maskedNumber = last4Digits.padStart(fullNumber.length, "*");
console.log(maskedNumber);                  // Expected output:
"************5581"

const str1 = "Breaded Mushrooms";
console.log(str1.padEnd(25, "."));  // Expected output: "Breaded
Mushrooms........"

const str2 = "200";
console.log(str2.padEnd(5));        // Expected output: "200  "
```

`` (backticks) - pour des expressions intelligentes

Utilisé avec des contenus chaîne de caractères :

```
const firstName = "Derek"
const verb = "develops"
const frequency = "daily"

// sans puis avec les backticks => beaucoup plus clair/propre/lisible
!!!
const name = firstName + " " + verb + " " + frequency
const literalName = `${firstName} ${verb} ${frequency}`
```

Utilisé pour éviter trop de caractères d'échappement :

```
Double quotes: "He said, \"Don't do that!\""
Single quotes: 'He said, "Don\'t do that!"'

// avec les backticks => beaucoup plus clair/propre/lisible !!!
Backtick literal: `He said, "Don't do that!"`
```

Utilisé lors de chaîne de caractères multi-lignes :

```
const myUglyString = "<div>\n<p>Hello world!</p>\n</div>";

const myBeautifulString = `<div>
    <p>Hello world!</p>
</div>`;
```

Cas concret d'utilisation :

```
const axios = require("axios")
const clientID = "YOUR_CLIENT_ID"
const secretID = "YOUR_SECRET_ID"
const params = `?client_id=${clientID}&client_secret=${secretID}`
function getProfile(username) {
  return
axios.get(`https://api.github.com/users/${username}${params}`).then(
    ({ data }) => data
    // equivalent to function(user) { return user.data }
  )
}
```

`new Set()` - pour supprimer les doublons

Pour produire un nouveau tableau sans les doublons éventuellement présents.

```
const dupes = [1, 2, 1, 3, 2, 4, 4, 2, 4, 3, 2, 1];
const uniqueSet = new Set( dupes );     // {1, 2, 3, 4}
const noDupes = [ ...uniqueSet ];       // Pour retransformer en un
tableau à l'aide de l'opérateur "spread"
```

Déclaration de fonction

Standard

```
function doStuff() {};
```

Sous forme d'expression de fonction

```
const doStuff = function() {}
```

Sous forme d'expression de fonction anonyme

```
const doStuff = () => {}
```

Fonctions immédiatement invoquées (IIFE) et lambda expressions

IIFE = Immediately Invoked Function Expressions.

Ces fonctions sont définies et **exécutées immédiatement**. Elles sont souvent utilisées pour créer un **contexte isolé** ou encapsuler du code sans polluer l'espace global.

```
(function(){ ... })()
```

ou

```
(() => { ... })()
```

Utilisation combinée de `map()` + `filter()` + `reduce()` + autres

On peut facilement combiner ces opérateurs en cascade.

```
data = [
  {
    name: 'Butters',
    age: 3,
    type: 'dog'
  },
  {
    name: 'Lizzy',
    age: 6,
    type: 'dog'
  },
  {
    name: 'Red',
    age: 1,
    type: 'cat'
  },
  {
    name: 'Joey',
    age: 3,
    type: 'dog'
  },
];

let ages = data.filter((animal) => {
    return animal.type === 'dog';
}).map((animal) => {
    return animal.age * 7
}).reduce((sum, animal) => {
    return sum + animal.age;
});
// ages = 84
```

Mieux et beaucoup plus lisible si les méthodes sont séparément créées/disponibles :

```
let isDog = (animal) => {
  return animal.type === 'dog';
}

let dogYears = (animal) => {
  return animal.age * 7;
}

let sum = (sum, animal) => {
  return sum + animal;
}

let ages = data
  .filter(isDog)
  .map(dogYears)
  .reduce(sum);
// ages = 84
```

Asynchronisme avec `async`, `await` et les `promises`

```
// Le choix d'une pizza et d'une boisson peut se faire en même temps
async function selectPizza() {
  const pizzaData = await getPizzaData();      // appel async
  const chosenPizza = choosePizza();           // appel sync
  await addPizzaToCart(chosenPizza);           // appel async
}

async function selectDrink() {
  const drinkData = await getDrinkData();      // appel async
  const chosenDrink = chooseDrink();           // appel sync
  await addDrinkToCart(chosenDrink);           // appel async
}

// Mais il faudra que les deux aient été réalisés pour pouvoir
procéder
// ensuite à la commande.
(async () => {
  const pizzaPromise = selectPizza();
  const drinkPromise = selectDrink();
  await pizzaPromise;
  await drinkPromise;
  orderItems();                                // appel async
})();

// Cette manière de faire est préférable !
Promise.all([selectPizza(), selectDrink()]).then(orderItems); //
appel async
```

Autre exemple :

```
// Obtient la liste des choses commandées et leur détails, le tout de
manière asynchrone.
async function orderItems() {
  const items = await getCartItems(); // appel async
  const noOfItems = items.length;
  const promises = [];

  for (var i = 0; i < noOfItems; i++) {
    const orderPromise = sendRequest(items[i]); // appel async
    promises.push(orderPromise);                // appel sync
  }

  await Promise.all(promises); // appel async
}

// Idem. Mais cette manière de faire est préférable !!
async function orderItems() {
  const items = await getCartItems(); // appel async
  const promises = items.map((item) => sendRequest(item));
  await Promise.all(promises); // appel async
}
```